

ICT703 Programming

Module 10 Pandas to manipulate data



WARNING NOTICE

Commonwealth of Australia | Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of the University of the Sunshine Coast pursuant to Part VA or Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act.
Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

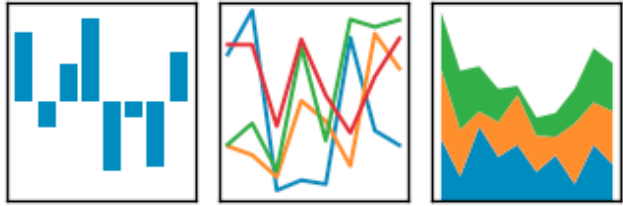


Today's Learning Objectives

- Continue raising the level of our programming
 - instead of updating one value at a time...
 - create/modify a whole list (1D) or **table (2D)** of values!
 - shrink dozens of lines of code down to a single line
- Learn Pandas **DataFrame** programming (2D data)

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



Pandas is a popular Python library for data science

- **Series** values for 1D sequences of data (like lists)
- **DataFrame** values for 2D tables of data
- Note: 3D, 4D,... tables are also possible, with *hierarchical indexes*, but this is beyond the scope of this course.

Pandas DataFrame

A DataFrame object is a 2D table of values

- like a spreadsheet
- has an *ordered* collection of columns
 - each column can have a different type
 - but values within the same column should have same type
- has an **index** for the rows (just like Series)
- has an **index** for the columns (column labels)

... column index ...

... row index ...

Creating a DataFrame

You can create a DataFrame from:

- a list of tuples (each tuple becomes a row)

```
houses = pandas.DataFrame([("NSW",2407.9,2823.6), ("VIC",1934.2,2243.0), ...])
```

- a Python dictionary containing lists

```
hdhouses = pandas.DataFrame({ "Internet":[2407.9,1934.2,...],  
                              "Households":[2823.6,2243.0,...]  
                              },  
                             index=['NSW', 'VIC',...])
```

- by reading a CSV file

Creating DataFrame from list-of-tuples

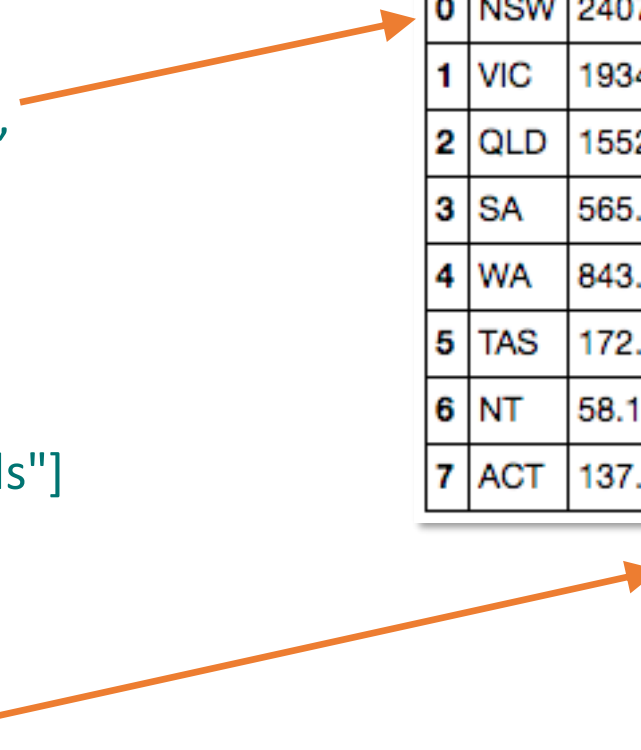
```
import pandas as pd
# 1. create it, with a list of tuples
houses = pd.DataFrame([("NSW",2407.9,2823.6),
                        ("VIC",1934.2,2243.0),
                        ... ])
```

```
# 2. set the column labels
houses.columns = ["State", "Internet", "Households"]
```

```
# 3. make the 'State' column to be the row-index
houses2 = houses.set_index("State")
```

houses			
	0	1	2
0	NSW	2407.9	2823.6
1	VIC	1934.2	2243.0
2	QLD	1552.4	1798.4
3	SA	565.1	
4	WA	843.6	
5	TAS	172.0	
6	NT	58.1	
7	ACT	137.2	

houses2		
	Internet	Households
State		
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7



Creating DataFrame from a dictionary

```
import pandas as pd
dhouses = pd.DataFrame({ "Internet":[2407.9,1934.2,...],
                        "Households":[2823.6,2243.0,...]},
                        index=['NSW', 'VIC',...])
```

```
#easily rearrange columns by creating a new DF
dhouses2 = dhouses[ ["Internet", "Households" ] ]
```

dhouses		
	Households	Internet
NSW	2823.6	2407.9
VIC	2243.0	1934.2
QLD	1798.4	1552.4
SA	685.4	
WA	958.0	
TAS	210.5	
NT	65.3	
ACT	145.7	

dhouses2		
	Internet	Households
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7

Read a DataFrame from a CSV file

This reads a CSV file with one header row (column labels)

The row index will be generated row numbers: 0, 1, 2, ...

```
frame = pd.read_csv("eg.csv")
```

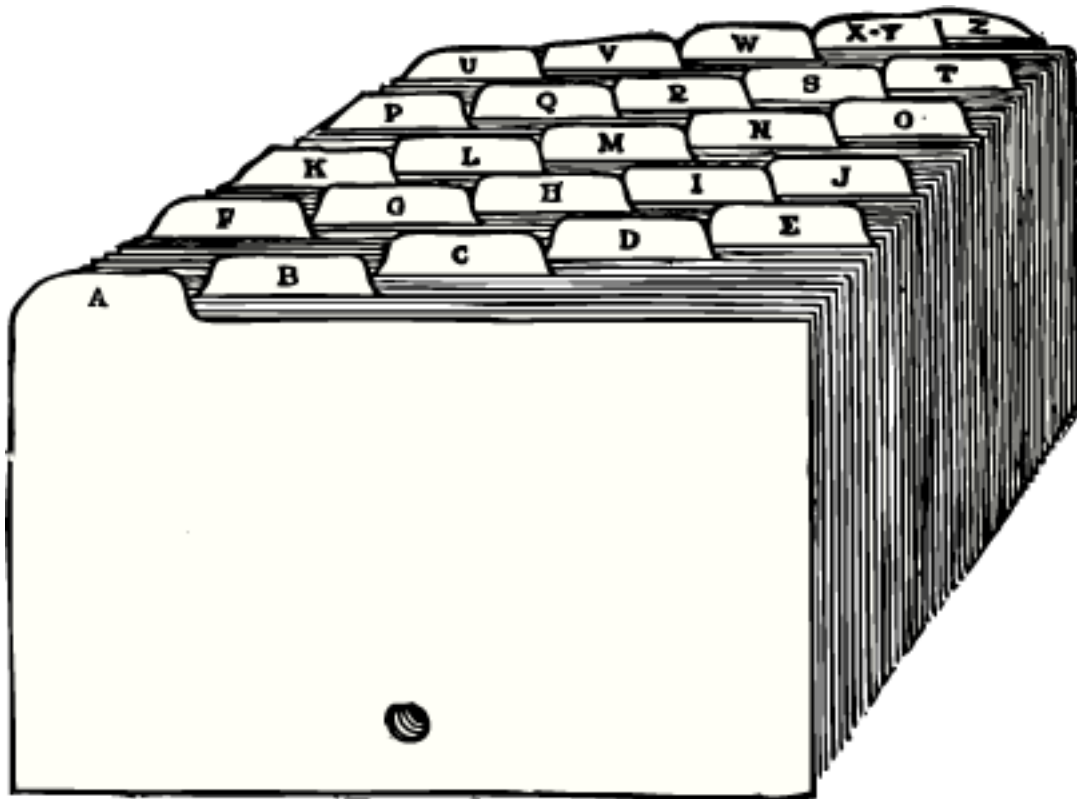
We can tell it to use a certain column for the row index.

```
frame = pd.read_csv("eg.csv", index_col="id")
```

If we have a data-only CSV file (no header row), we can give column labels

```
frame = pd.read_csv("eg.csv", header=None, names=["a", "b",...])
```

Indexing a DataFrame



[https://commons.wikimedia.org/wiki/File:Index_cards_\(tabbed,_showing_hole\).png](https://commons.wikimedia.org/wiki/File:Index_cards_(tabbed,_showing_hole).png)

But a DataFrame has *two* indexes:

- a column index, and
- a row index

Indexing a DataFrame: columns

- We can index it with a column label (returns a Series):

```
houses2["Internet"]
```

or the object-attribute shortcut:

```
houses2.Internet
```

```
NSW      2407.9
VIC      1934.2
QLD      1552.4
SA        565.1
WA        843.6
TAS       172.0
NT         58.1
ACT       137.2
Name: Internet, dtype: float64
```

- Multi-column index: use a list of column labels

```
houses2[["Internet", "Households", "Internet"]]
```

dhouses2

	Internet	Households
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7

	Internet	Households	Internet
NSW	2407.9	2823.6	2407.9
VIC	1934.2	2243.0	1934.2
QLD	1552.4	1798.4	1552.4
SA	565.1	685.4	565.1
WA	843.6	958.0	843.6
TAS	172.0	210.5	172.0
NT	58.1	65.3	58.1
ACT	137.2	145.7	137.2

Indexing a DataFrame: rows

- We index `houses2.loc[...]` with a row label

```
houses2.loc["NSW"]
```

```
Internet      2407.9
Households    2823.6
Name: NSW, dtype: float64
```

dhouses2

	Internet	Households
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7

- Multi-row index: use a list of row labels

```
eastern_states = ["QLD", "NSW", "VIC"]
houses2.loc[eastern_states]
```

	Internet	Households
QLD	1552.4	1798.4
NSW	2407.9	2823.6
VIC	1934.2	2243.0

- Slicing (same result, maybe different row order)

```
houses2.iloc[0:3]
```

```
houses2.loc["NSW":"QLD"]
```

WARNING: slicing indexes **rows**, not columns

WARNING: slicing with names *includes* the end value.

- Note: use `.loc(...)` for labels, `.iloc(...)` for positions

Indexing a DataFrame: cells

- We index with a column name then a row

```
houses2["Households"]["NSW"]
```

```
2823.6
```

or using the object-attribute shortcut:

```
houses2.Households["NSW"]
```

- We can index by *row first*, if we use .loc or .iloc

```
houses2.loc["NSW"].Internet
```

```
2407.9
```

dhouses2

	Internet	Households
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7

Summary of Indexing a DataFrame (df)

- You want to index by **labels**? Use .loc
`df.loc[ row_labels, col_labels ]`

`df[ col_labels ]` and `df.col_label` is same as `df.loc[ :, col_labels ]`

Note: `col_labels` can be `"col3":"col6"`. This includes both end points!

- You want to index by **positions or position-slices**? Use .iloc
`df.iloc[ row_positions, col_positions ]`
 - **But!**
 - **Shortcut 1:** `df[ row_slice ]` is same as `df.iloc[ row_slice, : ]`
 - **Shortcut 2:** `df[ bool_array ]` is same as `df.iloc[ bool_array, : ]`

Mutating a DataFrame



[https://commons.wikimedia.org/wiki/File:The_mutants!_\(9363719694\).jpg](https://commons.wikimedia.org/wiki/File:The_mutants!_(9363719694).jpg)

Mutate:

- update values
- add columns/rows
- drop columns/rows

Assigning *into* a DataFrame

- We can update (mutate) a single cell

```
houses2.Internet["ACT"] += 2.8
```

Note: **chained indexing**, `df[col][row]` is
bad practice on left hand side of assignment.

Instead use: `df.loc[row, col]`

Bad: `houses2["Internet"]["ACT"] += 2.8`

Good: `houses2.loc["ACT", "Internet"] += 2.8`

- We can update multiple rows or columns!

```
houses2.Internet[ ["SA", "WA", "TAS"] ] = 0.0
```

or equivalently

```
houses2.loc[ ["SA", "WA", "TAS"], "Internet" ] = 0.0
```

```
houses2.loc["QLD", : ] = [1600, 1900] # update the whole row for QLD
```

dhouses2

	Internet	Households
NSW	2407.9	2823.6
VIC	1934.2	2243.0
QLD	1552.4	1798.4
SA	565.1	685.4
WA	843.6	958.0
TAS	172.0	210.5
NT	58.1	65.3
ACT	137.2	145.7

Adding/Removing columns

- Add columns by assigning to a new column name

```
houses2["Percent"] = houses2.Internet / houses2.Households
```

Note: must use *df[c]* indexing on left-hand side to add new column *c*.
Cannot use *df.c* shorthand to create a *new* column

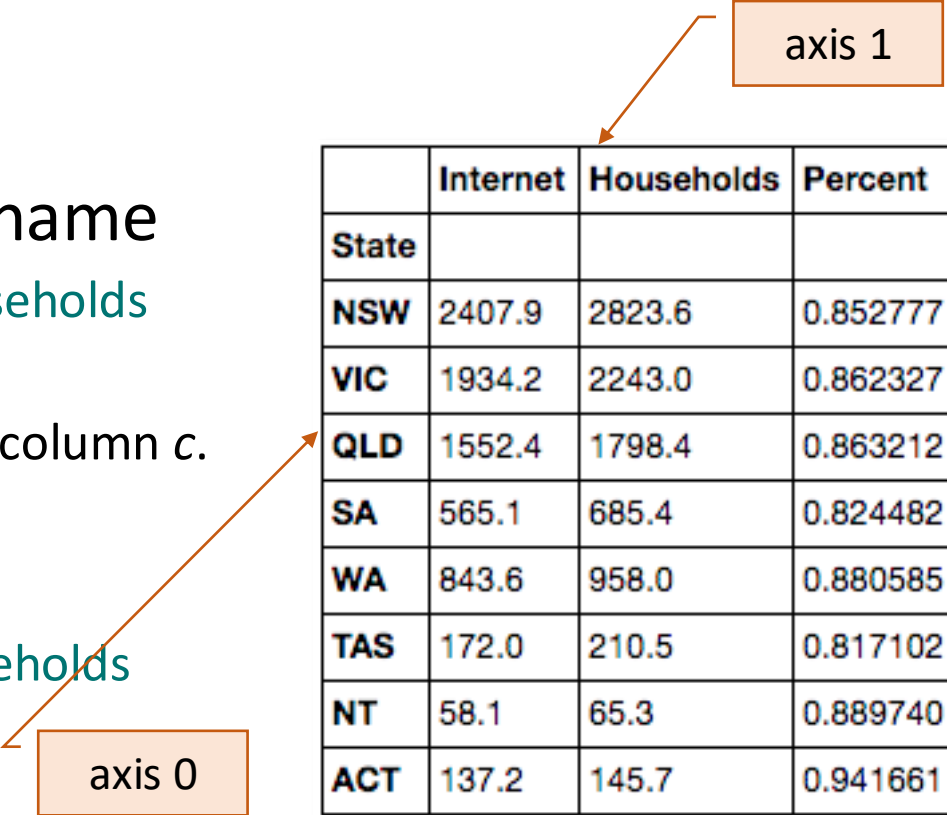
This will not work:

```
X houses2.Percent = houses2.Internet / houses2.Households
```

- To delete (drop) a column:

```
houses3 = houses2.drop("Percent", axis=1)
```

```
houses3 = houses2.drop(columns="Percent")
```



	Internet	Households	Percent
State			
NSW	2407.9	2823.6	0.852777
VIC	1934.2	2243.0	0.862327
QLD	1552.4	1798.4	0.863212
SA	565.1	685.4	0.824482
WA	843.6	958.0	0.880585
TAS	172.0	210.5	0.817102
NT	58.1	65.3	0.889740
ACT	137.2	145.7	0.941661

Adding/Removing rows

- Can add a row using `h3 = houses2.append(row)`

Where *row* is either:

- a dictionary (mapping column names to values)
- or a Series object.

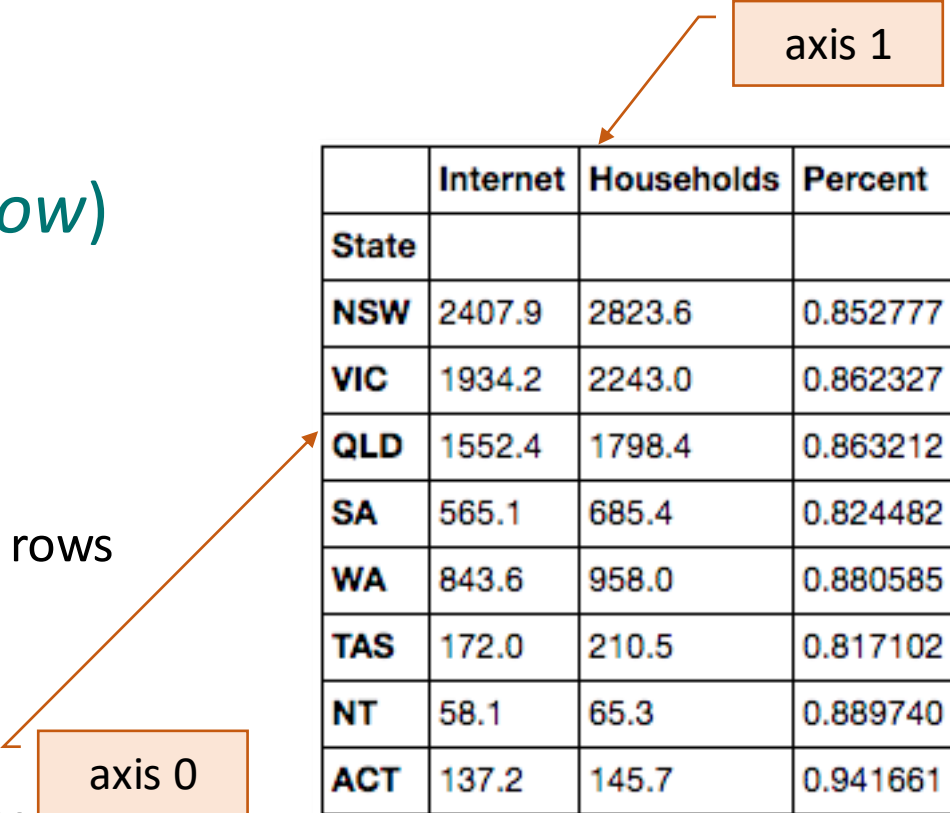
But append is slow, so it is better to read/create all the rows at the beginning if possible.

Do not add rows one by one.

- To delete (drop) a row (or a list or slice of rows):

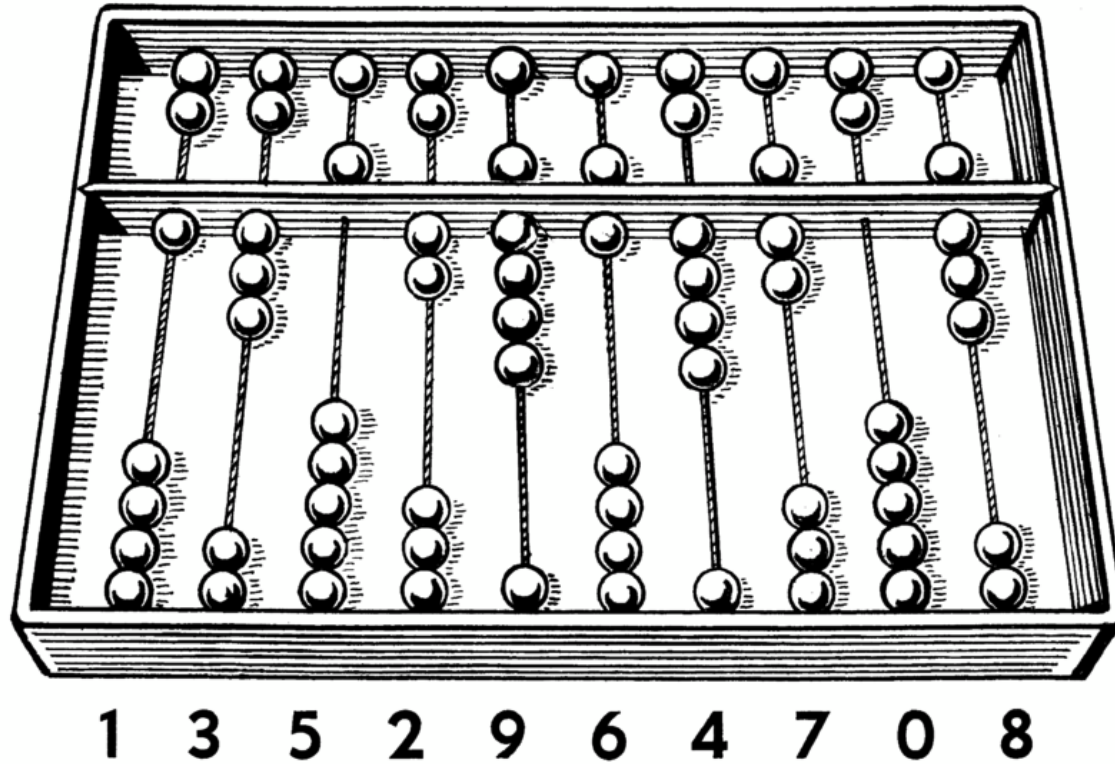
`houses3 = houses2.drop("TAS", axis=0)`

`houses3 = houses2.drop("TAS")`



	Internet	Households	Percent
State			
NSW	2407.9	2823.6	0.852777
VIC	1934.2	2243.0	0.862327
QLD	1552.4	1798.4	0.863212
SA	565.1	685.4	0.824482
WA	843.6	958.0	0.880585
TAS	172.0	210.5	0.817102
NT	58.1	65.3	0.889740
ACT	137.2	145.7	0.941661

Calculating with DataFrames



[https://commons.wikimedia.org/wiki/File:Abacus_1_\(PSF\).png](https://commons.wikimedia.org/wiki/File:Abacus_1_(PSF).png)

1. Calculating across rows
 - return a new column (Series)
 - or add a new column
2. Calculating down columns
 - (reduce functions)

Calculating across a DataFrame

- We calculate with columns just like ordinary arithmetic

`result = houses2.Internet / houses2.Households`

Note:

- this gives a Pandas **Series** object (a column)
- it automatically aligns the row indexes
- the 100 scalar constant is *broadcasted* to all the rows

	Internet	Households	Percent
State			
NSW	2407.9	2823.6	0.852777
VIC	1934.2	2243.0	0.862327
QLD	1552.4	1798.4	0.863212
SA	565.1	685.4	0.824482
WA	843.6	958.0	0.880585
TAS	172.0	210.5	0.817102
NT	58.1	65.3	0.889740
ACT	137.2	145.7	0.941661

- We often add result columns into our existing DataFrame

`houses2["Percent"] = houses2.Internet / houses2.Households`



Calculating across a DataFrame 2

- We can also merge DataFrames, or combine DataFrame+Series

- If they have similar row indexes

- If we have a Series on the population of Aussie states

```
pop = pd.Series([7.6, 5.9, 4.8, 2.6, 1.7, 0.5]) # in millions
```

```
pop.index = ["NSW", "VIC", "QLD", "WA", "SA", "TAS"] # only 6 states!
```

```
pop *= 1000 # now in thousands
```

- Can we calculate the average household size now?

```
avsize = pop / houses2.Households
```

- Yes!

- And it was smart enough to align the states correctly.
 - The rows with missing population data give **NaN** (*Not a Number*)
 - Save it into our DataFrame: `houses2["Average"] = avsize`

avsize

ACT	NaN
NSW	2.691599
NT	NaN
QLD	2.669039
SA	2.480303
TAS	2.375297
VIC	2.630406
WA	2.713987
dtype: float64	

Viewing Column information

<code>df.col.describe()</code>	<code>#summary of values in a column</code>
<code>df.col.values()</code>	<code>#will list all the values in the column</code>
<code>df.col.unique()</code>	<code>#will list only unique values in the column</code>
<code>df.col.value_counts()</code>	<code>#will count the frequency of each value</code>

value_counts of the 'Crash_Type' column

In: `crashes.Crash_Type.value_counts()`

Out: Single vehicle 17818
Multiple vehicle 17125
Pedestrian 7373

Only three kinds of crash type

Calculating down a DataFrame

Internet	958.812500
Households	1116.237500
Percent	86.648562
Population	3850.000000
Average	2.593439
dtype: float64	

- A column is just a Pandas Series
 - so we use Series *reduce functions* down a column: `table[col].func()`
 - or even all columns at once: `houses2.mean()`

- Useful reduce functions:

<code>houses2.Average.count()</code>	→ 6	# number of <i>non-NaN</i> values
<code>houses2.Average.sum()</code>	→ 15.56	# sum of all the <i>non-NaN</i> values
<code>houses2.Average.mean()</code>	→ 2.59	# average (mean) of non-NaN values
<code>houses2.Average.median()</code>	→ 2.6497	# median (middle) value of non-NaN...
<code>houses2.Average.std()</code>	→ 0.135	# the standard deviation of non-NaN...
<code>houses2.Average.max()</code>	→ 2.71	# the maximum value
<code>houses2.Average.idxmax()</code>	→ "WA"	# index of the max value

Data Prep.3: Merging Data

- The table has several same-name columns, but we merge on CrashID

```
data = crashes3.merge(deaths, on="CrashID", suffixes=("", "_2"))
```

crashes3

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Num
0	4201601010001	SA	1	1	2016	1	0	Single vehicle	1
1	2201601010001	VIC	1	1	2016	20	30	Single vehicle	1
2	1201601020001	NSW	2	1	2016	0	30	Single vehicle	1

deaths

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type
0	4201601010001	SA	1	1	2016	1	0	Single
1	2201601010001	VIC	1	1	2016	20	30	Single
2	1201601020001	NSW	2	1	2016	0	30	Single
3	5201601020001	WA	2	1	2016	17	20	Multiple

data

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Num	...	...	Heavy_Rigid_Truck_Involvement_2	Articulated_Truck_Involvement_2	Speed_Limit_2	Road_User	Gender	Age
0	4201601010001	SA	1	1	2016	1	0	Single vehicle	1	...	...	No	No	110	Driver	Male	40
1	2201601010001	VIC	1	1	2016	20	30	Single vehicle	1	...	...	No	No	80	Motor cycle rider	Male	25
2	1201601020001	NSW	2	1	2016	0	30	Single vehicle	1	...	...	No	No	100	Passenger	Male	18

Data Grouping

- **groupby(...)** supports many kinds of grouping
 - `groupby(column)` - by a single *column*
 - `groupby([col1,col2])` – by *col1*, then by *col2*
 - `groupby(function)` – by *function* applied to indexes

`df.groupby(col1).mean()` #this will group by col1 and average **all** numeric columns

`df.groupby(col1)[col2].mean()` #this will group by col1 and average **only** col2



Custom Calculations/Code

- You can easily use Python to loop over all columns:

```
for col in houses2.columns:           # or just houses2
    print(col, houses2[col].QLD)
```

Internet 1552.4
Households 1798.4
Percent 86.3211743772
Average 2.669039

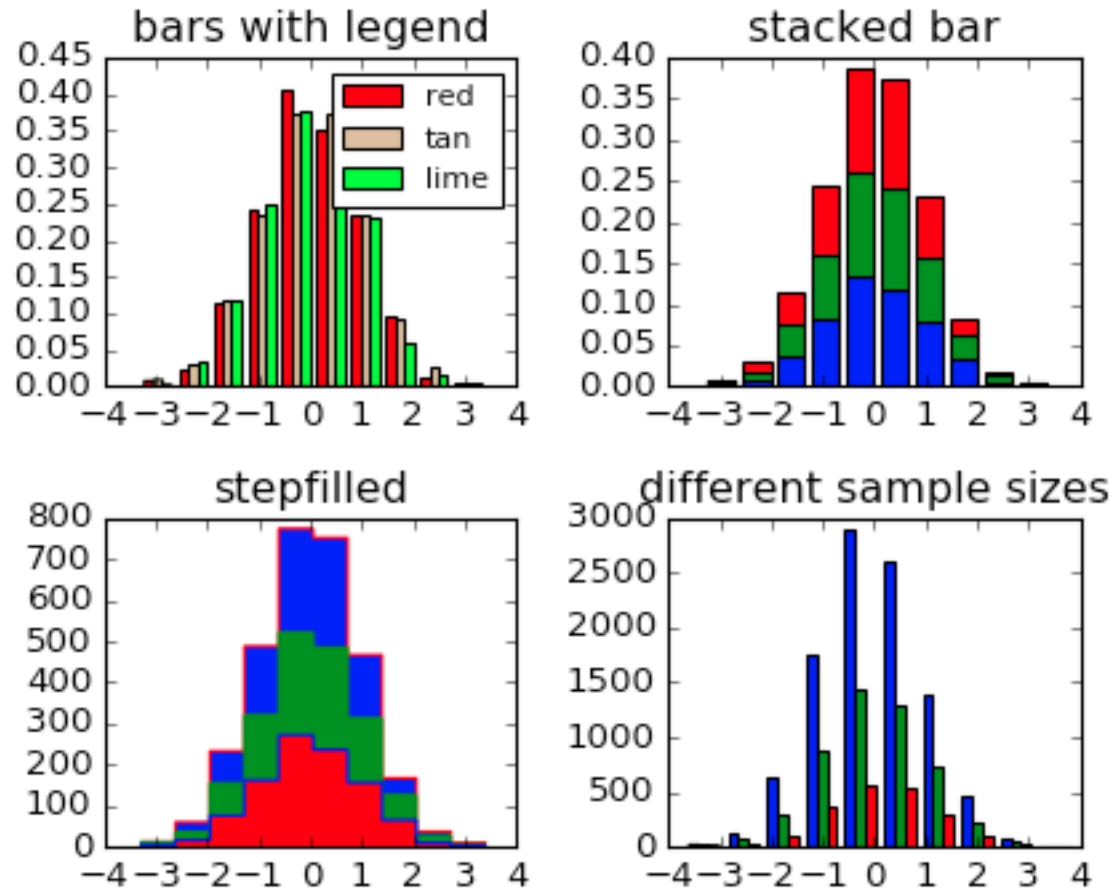
- Or to loop over all rows:

```
for state in houses2.index:
    print(state, houses2.Percent[state], "% have internet access.")
```

NSW 85.3% have internet access.
VIC 86.2% have internet access.
QLD 86.3% have internet access.
SA 82.4% have internet access.
WA 88.1% have internet access.
TAS 81.7% have internet access.
NT 89.0% have internet access.
ACT 94.2% have internet access.

- So it is easy to do your own custom calculations
 - use loops and if-else statements

Visualising DataFrames



- Plotting can be done using matplotlib
- Or through Pandas

Using Matplotlib

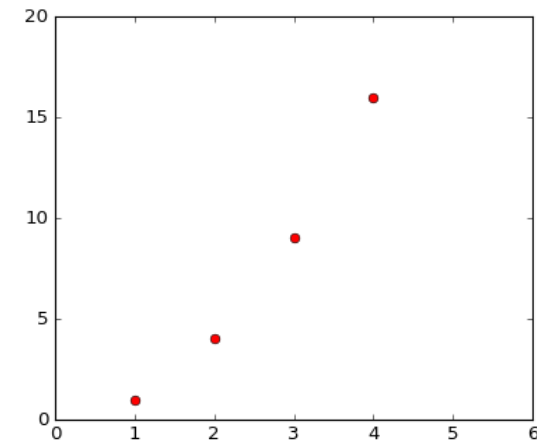
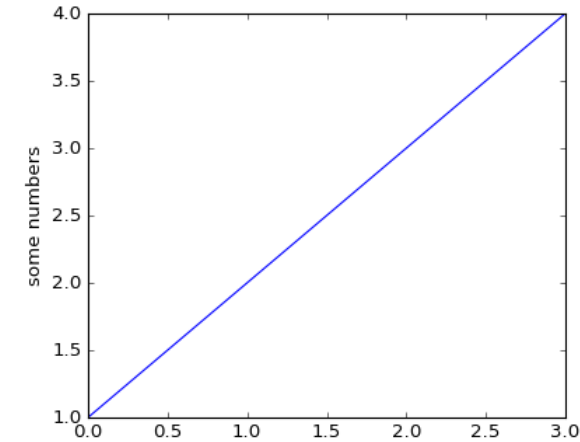
Second 'line plot' example?

```
import matplotlib.pyplot as plt  
plt.plot([1,2,3,4])  
plt.ylabel('some numbers')  
plt.show()
```

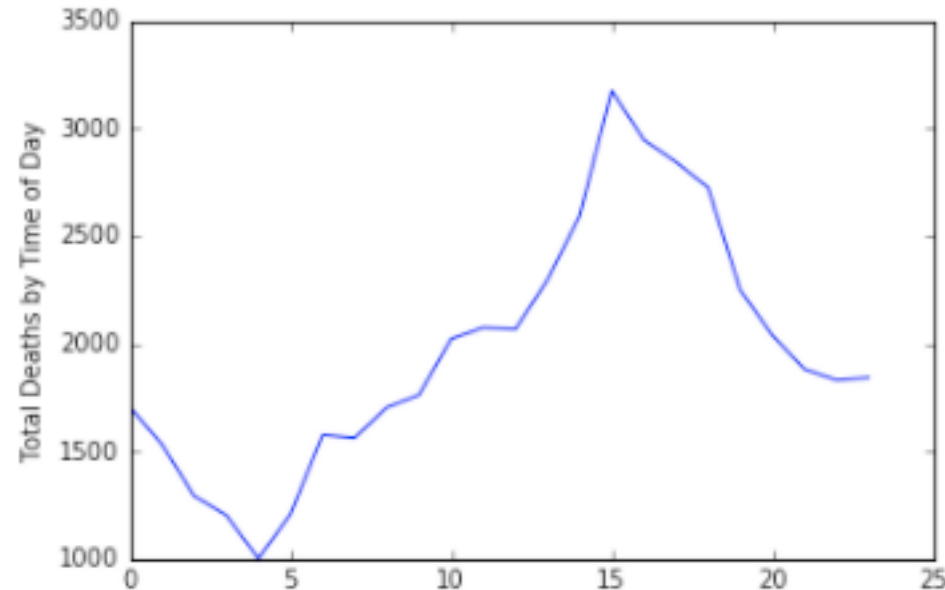
Or the next example, with more features?

```
import matplotlib.pyplot as plt  
plt.plot([1,2,3,4], [1,4,9,16], 'ro')  
plt.axis([0, 6, 0, 20])  
plt.show()
```

[From: http://matplotlib.org/users/pyplot_tutorial.html]



```
In [9]: plt.plot(hours, deaths)
plt.ylabel('Total Deaths by Time of Day')
plt.show()
# or: plt.savefig("deaths_per_hour.png")
```



Surprising!

3-4 pm is the most dangerous time on the roads!

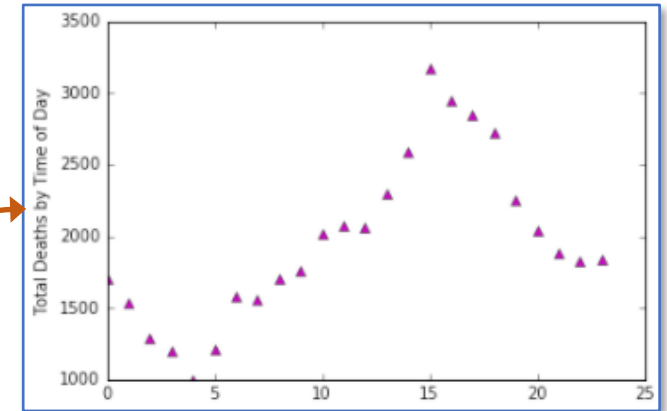
Or maybe it is just the time when more cars are on the roads?

Customising...

- To learn more about options of `plt.plot` we type:
`plt.plot?`
- We see that a third argument can specify:
"<colour><linestyle>"

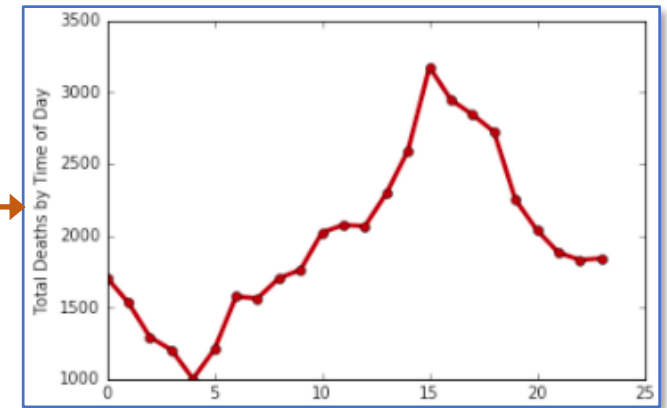
Eg 1 "r"

```
plt.plot(hours, deaths, "m^")  
plt.ylabel('Total Deaths by Time of Day')  
plt.show()
```



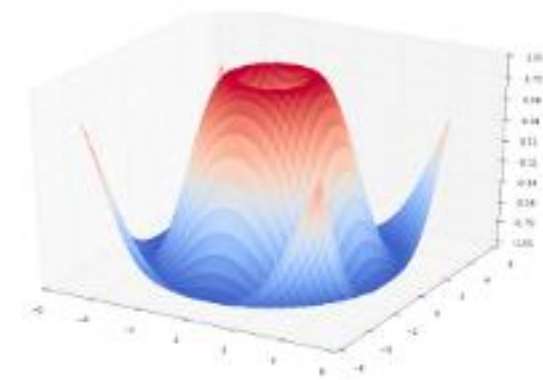
Eg 2 or separate keyword arguments:

```
plt.plot(hours, deaths, color="#AA0000", marker="o", linewidth=3)  
plt.ylabel('Total Deaths by Time of Day')  
plt.show()
```



Other useful graphs

- Line plots with error bars
 - `plt.errorbar(x_vals, y_vals, yerr=y_errors)`
- Scatter plots:
 - `plt.scatter(x_values, y_values)`
- Vertical and Horizontal bar plots:
 - `plt.bar(x_positions, y_heights)`
 - `plt.barh(y_positions, x_widths)`
- And many others...
 - See official docs: <http://matplotlib.org>
 - or type `plt.bar?` etc. to get help...



Using Pandas Plot

Scatter Plot

We want to plot Internet access vs Household size

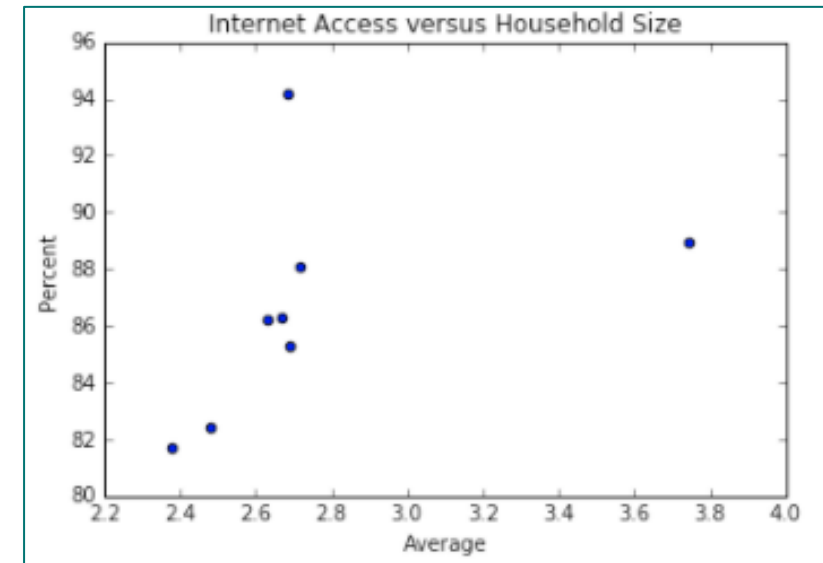
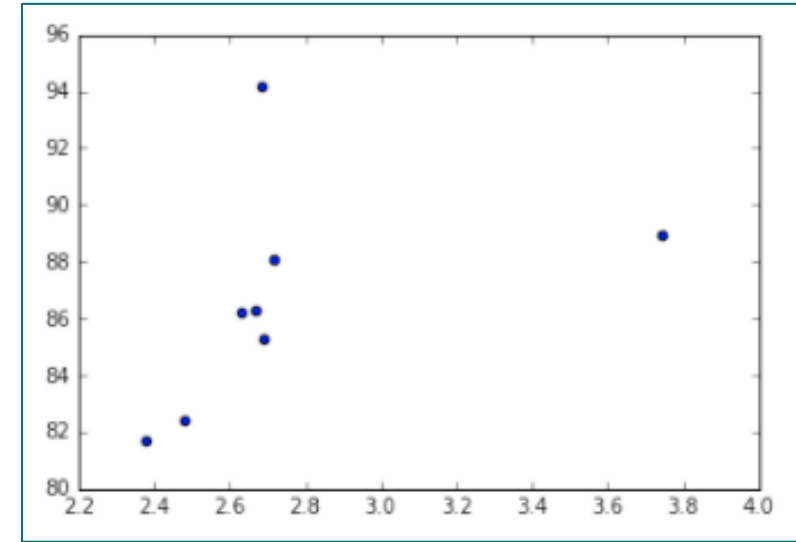
- We can do this with just matplotlib:

```
import matplotlib.pyplot as plt
```

```
plt.scatter(houses2.Average, houses2.Percent)
```

- Or we can ask our DataFrame to plot itself:

```
houses2.plot("Average", "Percent", kind="scatter",  
            title="Internet Access versus Household Size")
```

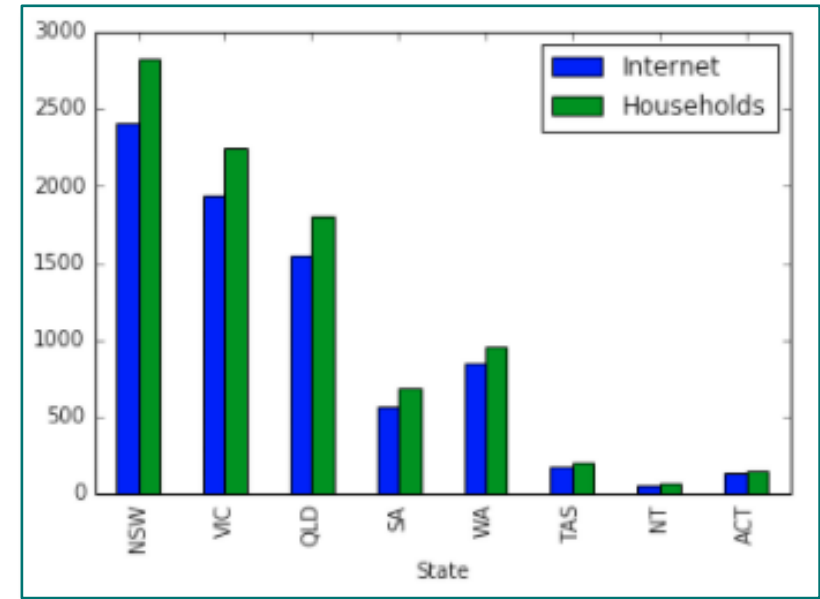


Graphing columns is easy!

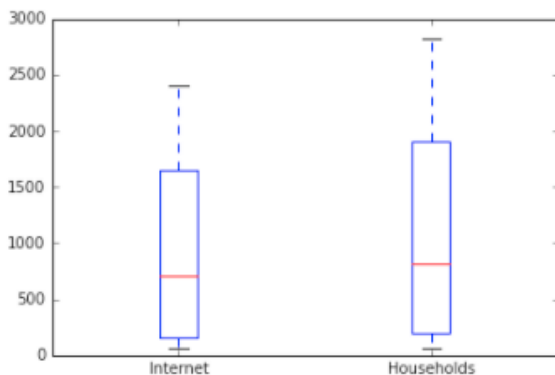
- Let's graph two columns

```
houses2.plot(y=["Internet","Households"], kind="bar")
```

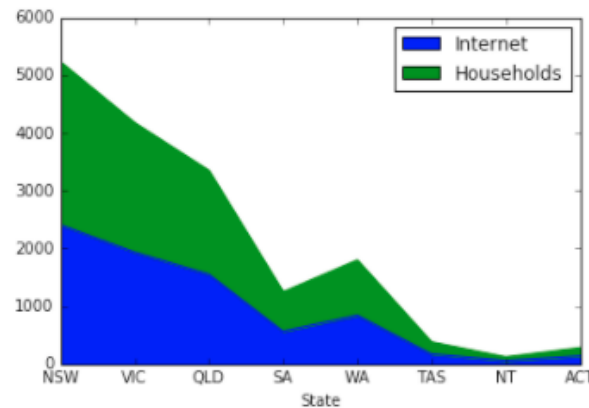
Note: x=None, so it defaults to the row index (States)



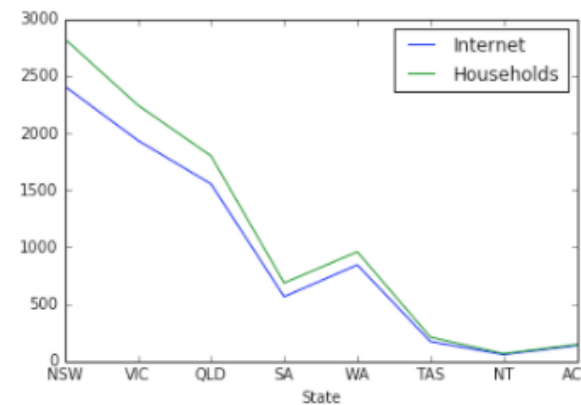
kind="bar"



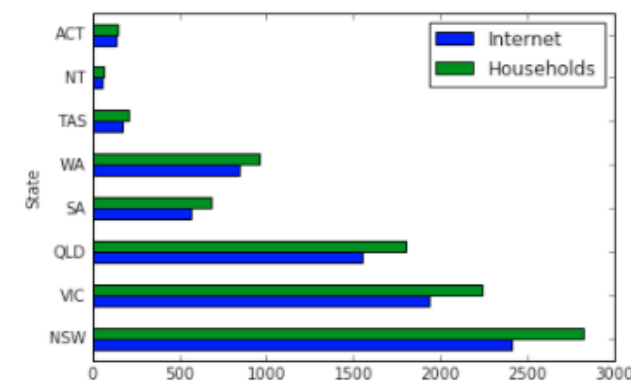
kind="box"



kind="area"



kind="line"



kind="barh"

Conclusions

Pandas DataFrames:

- are 2D tables of values
 - plus optional *row-index* and *column-index*
- very easy to read a DataFrame from a CSV file
- lots of powerful query and reduce functions
- easy to calculate new columns
 - Add hundreds/thousands of 'Excel formulae' in a single line of Python code!
- automatically aligns the indexes when you combine DataFrames & Series
- integrates nicely with matplotlib to get useful visualisations